

Loop Satatement in C

- **Loops** in programming are used to repeat a block of code until the specified condition is met.
- A **loop statement** allows programmers to execute a statement or group of statements multiple times without repetition of code.

Example to illustrate need of loop

```
// C program to illustrate need of loops
#include <stdio.h>

int main()
{
    printf "Hello World\n"
    printf "Hello World\n"
    printf "Hello World\n"
    printf "Hello World\n"
    printf "Hello World\n"
    printf "Hello World\n"
    return 0;
}
```

Output:

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

There are mainly two types of loops in C Programming:

➔ **Entry Controlled loops:** In Entry controlled loops the test condition is checked before entering the main body of the loop. **For Loop and While Loop** is Entry-controlled loops.

➔ **Exit Controlled loops:** In Exit controlled loops the test condition is evaluated at the end of the loop body. The loop body will execute at least once, irrespective of whether the condition is true or false. **do-while Loop** is Exit Controlled loop.

Loop Type	Description
for loop	first Initializes, then condition check, then executes the body and at last, the update is done.
while loop	first Initializes, then condition checks, and then executes the body, and updating can be inside the body.
do-while loop	do-while first executes the body and then the condition check is done.

➤ for Loop

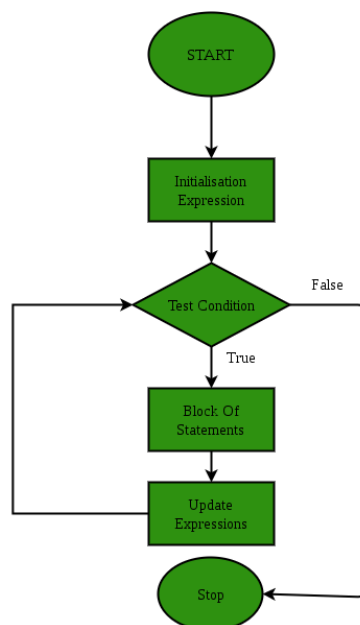
- The **for loop** in C Language provides a functionality/feature to repeat a set of statements a defined number of times.
- The for loop is in itself a form of an **entry-controlled loop**.
- Unlike the while loop and do...while loop, the for loop contains the initialization, condition, and updating statements as part of its syntax. It is mainly used to traverse arrays, vectors, and other data structures.
- “for loop in C programming is a repetition control structure that allows programmers to write a loop that will be executed a specific number of

times. for loop enables programmers to perform n number of steps together in a single line.”

Syntax:

```
for (initialize expression; test expression; update expression)
{
    //
    // body of for loop
    //
}
```

- In for loop, a loop variable is used to control the loop. Firstly we initialize the loop variable with some value, then check its test condition.
- If the statement is true then control will move to the body and the body of for loop will be executed. Steps will be repeated till the exit condition becomes true. If the test condition will be false then it will stop.



Example

```
// C program to illustrate for loop
#include <stdio.h>

// Driver code
```

```
int main()
{
    int i = 0;

    for (i = 1; i <= 10; i++)
    {
        printf "Hello World\n"
    }
    return 0;
}
```

Output:

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

➤ While Loop

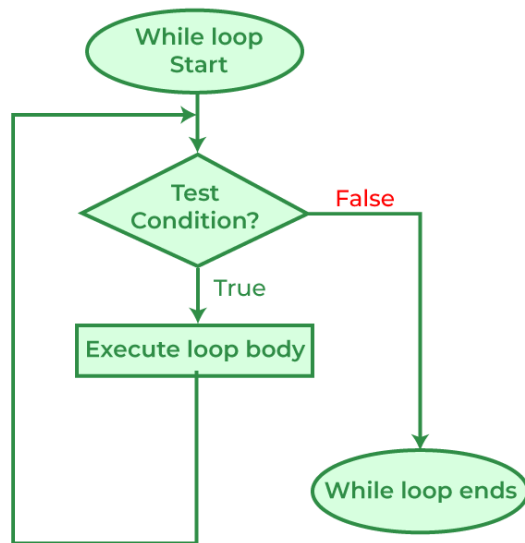
- While loop does not depend upon the number of iterations. In for loop the number of iterations was previously known to us but in the While loop, the execution is terminated on the basis of the test condition.
- If the test condition will become false then it will break from the while loop else body will be executed.

Syntax:

```
initialization_expression;

while (test_expression)
```

```
{  
    // body of the while loop  
  
    update_expression;  
}
```



Example

```
#include <stdio.h>  
// Driver code  
int main()  
{  
    int i = 2;  
    while(i < 10)  
    {  
        printf( "Hello World\n");  
        i++;  
    }  
    return 0;  
}
```

Output:

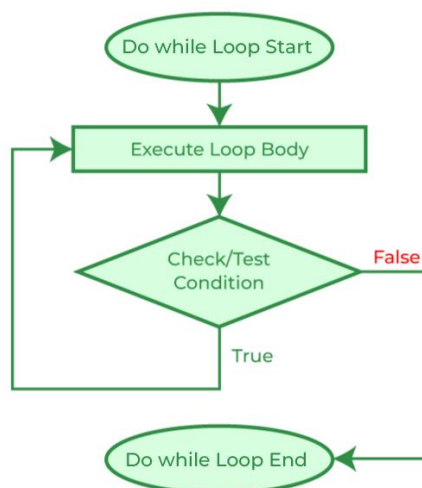
```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

➤ do-while Loop

- The do-while loop is similar to a while loop but the only difference lies in the do-while loop test condition which is tested at the end of the body.
- In the do-while loop, the loop body will **execute at least once** irrespective of the test condition.

Syntax:

```
initialization_expression;
do
{
    // body of do-while loop
    update_expression;
} while (test_expression);
```



Example:

```
#include <stdio.h>

int main()
{
    int i = 2;

    do
    {
        printf( "Hello World\n");

        i++;

    } while (i < 1);

    return 0;
}
```

Output: Hello World

Above program will evaluate ($i < 1$) as false since $i = 2$. But still, as it is a do-while loop the body will be executed once.

➤ Infinite Loop

- An infinite loop is executed when the test expression never becomes false and the body of the loop is executed repeatedly.
- A program is stuck in an Infinite loop when the condition is always true.
- Mostly this is an error that can be resolved by using Loop Control statements.